AUTOMATED SQL INJECTION DETECTION AND PREVENTION

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

ITA1416- ETHICAL HACKING

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

K. Nitheesh Kumar (192325090)

S. Sham (192325076)

T. Srikanth (192365030)

K. Sai Kumar (192372080)

SK. Mohammad Irfan (192472190)

Under the Supervision of

Dr.K.Ramesh Kumar

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled **"AUTOMATED SQL INJECTION DETECTION AND PREVENTION"** has been carried out by **K. Nitheesh Kumar (192325090), S. Sham (192325076), T. Sreekanth (192365030), K. Sai Kumar (192372080), SK. Mohammad Irfan (192472190)** under the supervision of **Dr.K.Ramesh Kumar** and is submitted in partial fulfilment of the requirements for the current semester of the B.E./B.Tech COMPUTER SCIENCE AND ENGINEERING programme, in the course **ITA1416-ETHICAL HACKING**, at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR
Dr. S PARTHIBAN
PROFESSOR
COMPUTER SCIENCE AND ENGINEERING
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY
Dr. K. Ramesh Kumar,
PROFESSOR
NEXT-GEN COMPUTING
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

We, **SHAM S.,** of the COMPUTER SCIENCE AND ENGINEERING Department, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled **"AUTOMATED SQL INJECTION DETECTION AND PREVENTION"** is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 02.09.2026

Signature of the Students with Names:

SHAM S. (192325076)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
K. Nitheesh Kumar (192325090)	Overall coordination; hybrid detection-engine design	Designed system architecture; developed the signature-matching module	Led penetration-test simulations using SQLMap/DVWA	Chapters 1, 3	18%
S. Sham (192325076)	Machine-learning classifier development	Implemented and trained the Random Forest / feature-extraction pipeline	Performed model evaluation (accuracy, precision, recall, F1)	Chapters 2, 4 (4.4-4.5)	20%
T. Sreekanth (192365030)	Web application & middleware integration	Built the Flask middleware and query-sanitization module	Conducted functional and regression testing	Chapter 4 (4.1-4.3)	20%
K. Sai Kumar (192372080)	Dataset preparation & literature survey	Curated and pre-processed the labelled SQL-query dataset	Cross-validated results and prepared confusion-matrix analysis	Chapter 2	19%
SK. Mohammad Irfan (192372190)	Documentation, risk analysis & project management	Prepared risk register, Gantt chart and requirement tables	Verified test cases against acceptance criteria	Chapters 3 (3.7), 5	23%
Total					100%

ABSTRACT

Structured Query Language (SQL) injection remains one of the most prevalent and damaging vulnerabilities affecting database-driven web applications, consistently ranked among the OWASP Top 10 security risks. Attackers exploit poorly validated user inputs to inject malicious SQL fragments, enabling unauthorized data access, data manipulation, authentication bypass, and complete database compromise. Existing defences such as static Web Application Firewalls (WAFs) and manually written signature rules struggle to keep pace with obfuscated and evolving attack patterns, while purely machine-learning-based detectors often report unacceptably high false-positive rates and add latency that is unsuitable for production traffic. This capstone project addresses this engineering problem by designing, implementing and evaluating an Automated SQL Injection Detection and Prevention system that combines lightweight signature-based pattern matching with a supervised machine-learning classifier in a single hybrid detection pipeline. The proposed system is implemented as a request-interception middleware that captures incoming HTTP parameters, tokenizes and extracts lexical and statistical features (special-character ratio, SQL-keyword density, comment/encoding indicators), performs a fast regular-expression signature check, and, for inputs that pass the signature stage, applies a trained Random Forest classifier to compute a maliciousness score. Requests classified as malicious are blocked and logged, while benign requests are forwarded through a parameterized-query layer that enforces safe execution on the backend MySQL database. The system was trained and validated on a labelled dataset of over 30,000 benign and malicious SQL query samples and tested against live attack simulations using the Damn Vulnerable Web Application (DVWA) and SQLMap. Experimental results demonstrate that the hybrid approach achieves 97.6% detection accuracy with a false-positive rate of 2.4%, outperforming standalone signature-based (86.4% accuracy, 14.2% FPR) and standalone ML-based (92.1% accuracy, 8.7% FPR) approaches, while maintaining an average per-request detection latency of approximately 31 milliseconds, well within acceptable real-time thresholds. The principal conclusion of this work is that combining deterministic signature matching with adaptive statistical learning yields a detection and prevention mechanism that is simultaneously more accurate, more robust to novel obfuscation techniques, and practically deployable as middleware without requiring changes to existing application logic.

Keywords: *SQL Injection, Intrusion Detection, Machine Learning, Web Application Security, Signature Matching, Parameterized Queries, OWASP Top 1*

TABLE OF CONTENTS

CHAPTERS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
SQLi	SQL Injection	-
WAF	Web Application Firewall	-
ML	Machine Learning	-
RF	Random Forest (classifier)	-
OWASP	Open Web Application Security Project	-
FPR	False Positive Rate	%
FNR	False Negative Rate	%
HTTP	Hypertext Transfer Protocol	-
DVWA	Damn Vulnerable Web Application	-
API	Application Programming Interface	-
ASVS	Application Security Verification Standard	-
DPDP	Digital Personal Data Protection (Act, 2023)	-
ms	Millisecond	ms

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Relational database systems form the backbone of nearly every modern web application, storing everything from user credentials to financial transactions. Structured Query Language (SQL) is the standard interface through which application logic communicates with these databases. When user-supplied input is concatenated directly into SQL statements without proper validation or parameterization, an attacker can alter the intended structure of a query, a class of vulnerability known as SQL Injection (SQLi). SQL injection has remained in the OWASP Top 10 list of web application security risks for over a decade and continues to be a leading cause of large-scale data breaches, including incidents affecting financial institutions, e-commerce platforms and government portals. The technical and application context of this project is a typical three-tier web architecture, comprising a client browser, an application server (built using Flask/PHP), and a backend MySQL database, which is representative of the majority of small and medium-scale web deployments that are most frequently targeted by automated SQLi scanners such as SQLMap and Havij.

1.2 Need for the Project

Traditional perimeter defences such as static Web Application Firewalls (WAFs) rely on hand-written regular-expression signatures. While effective against known attack strings, they are easily bypassed through encoding, case manipulation, inline comments and logically equivalent query rewriting. Purely machine-learning-based detectors, on the other hand, can generalize to novel obfuscation patterns but are frequently reported in the literature to suffer from high false-positive rates, misclassifying legitimate but unusual queries as malicious, which degrades user experience and erodes administrator trust. Small and medium enterprises, whose developers may not have dedicated security expertise, are disproportionately affected because they cannot afford commercial, cloud-based WAF subscriptions. This creates a clear engineering need for a lightweight, self-hostable, hybrid detection mechanism that combines the speed and precision of signature matching with the adaptability of statistical learning, without requiring changes to existing application source code.

- Why the problem needs to be addressed: SQL injection remains a top-ranked, high-impact vulnerability class with severe confidentiality, integrity and availability consequences.

- Who is affected: end users (data privacy loss), application owners (financial and reputational damage), and database administrators (recovery and compliance burden).
- Existing limitations: static signatures are brittle; pure ML approaches are costly to retrain and produce false positives; commercial WAFs are expensive and often opaque.
- Engineering significance: a hybrid, explainable, low-latency detection layer can be deployed as middleware, offering a practical and reproducible academic contribution.

1.3 Problem Statement

Design and implement an automated system capable of detecting and preventing SQL injection attacks in real time within a web-based database application, given the following interacting and often conflicting requirements: (i) the system must achieve high detection accuracy across multiple SQLi attack classes (union-based, boolean-based blind, time-based blind, error-based and stacked-query injection); (ii) it must keep false-positive rates low enough not to disrupt legitimate application traffic; (iii) it must operate within strict latency budgets suitable for real-time web request handling; (iv) it must be deployable as a middleware layer without requiring extensive modification of existing application code; and (v) it must remain resilient against obfuscation techniques such as inline comments, alternate encodings and case-randomisation used by attackers to evade signature-based defences.

1.4 Project Aim

To design and implement an automated hybrid SQL injection detection and prevention system that identifies malicious SQL queries in real time and blocks them before they reach the backend database, while keeping false positives and processing latency within acceptable engineering limits.

1.5 Project Objectives

- To study existing SQL injection attack vectors, obfuscation techniques and current detection/prevention approaches.
- To design a hybrid detection architecture combining regular-expression-based signature matching with a supervised machine-learning classifier.
- To develop a request-interception middleware module that can be integrated with Flask/PHP web applications with minimal code changes.
- To implement an input-sanitisation and parameterised-query enforcement layer that guarantees safe query execution for legitimate requests.

- To train and test the system against a labelled dataset of SQL queries and against live attack simulations using DVWA and SQLMap.
- To evaluate system performance in terms of accuracy, precision, recall, F1-score, false-positive rate and per-request detection latency.

Table 1.1: Project Objectives Summary

No.	Objective
1	To study existing SQL injection attack vectors and detection techniques
2	To design a hybrid detection architecture (signature + ML)
3	To develop an integrable middleware module
4	To implement input sanitisation and parameterised query enforcement
5	To test the system against a labelled dataset and live attack simulations
6	To evaluate accuracy, precision, recall, F1-score and latency

1.6 Scope

Included: Web applications using MySQL/PostgreSQL backends; detection of union-based, boolean-based blind, time-based blind, error-based and stacked-query SQL injection delivered through HTTP GET/POST parameters and headers.

Excluded: NoSQL injection (e.g., MongoDB operator injection), native mobile-application SQLi, physical/network-layer security, and denial-of-service attacks not related to SQL injection.

Assumptions: The attacker interacts with the system exclusively through the web front end; the backend database is MySQL 8.0; the middleware has access to raw HTTP request parameters before they reach the query-construction layer.

Boundary Conditions: The prototype was developed and tested on a Node.js/Flask demonstration application integrated with the Damn Vulnerable Web Application (DVWA) running on a local MySQL instance; production-scale distributed deployment was outside the project timeline.

1.7 Expected Engineering Outcome

The project delivers a working software prototype comprising: (a) a trained hybrid detection engine (signature module + Random Forest classifier) packaged as a Python/Flask middleware library; (b) a query-sanitisation and parameterisation layer; (c) a logging and alerting dashboard for detected attack attempts; and (d) an evaluation report quantifying detection accuracy, false-positive rate and latency against benchmark datasets and simulated attacks.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature, standards and existing products were identified through a structured search of IEEE Xplore, ACM Digital Library, ScienceDirect and Google Scholar using the keywords "SQL injection detection", "machine learning web application firewall", "signature-based intrusion detection" and "hybrid SQLi prevention", restricted to publications from 2015-2025. In addition, the OWASP Foundation's official documentation (OWASP Top 10, OWASP ASVS) and product documentation of commercial WAFs (ModSecurity, Cloudflare WAF, AWS WAF) were reviewed to understand industry practice. A total of 15 sources were shortlisted for detailed review based on relevance to hybrid or ML-assisted SQLi detection.

2.2 Review of Existing Work

Signature-based systems such as ModSecurity with the OWASP Core Rule Set rely on curated regular-expression rules and are widely deployed due to their low latency and interpretability, but multiple studies confirm they can be bypassed using encoding tricks, whitespace substitution and inline SQL comments. Anomaly-based approaches model normal query structure (e.g., using query length, token distribution) and flag deviations, offering some resilience to novel attacks at the cost of higher false-positive rates on legitimate but unusual queries. Machine-learning-based approaches reported in the literature include Support Vector Machines, Naive Bayes, Random Forest and, more recently, LSTM/CNN-based deep-learning classifiers trained on tokenised SQL query features; these achieve high accuracy on benchmark datasets but typically require substantial labelled training data and periodic retraining to remain effective as attack patterns evolve. Commercial cloud WAFs (Cloudflare, AWS WAF, Azure WAF) combine proprietary signature sets with managed rule updates and, in some tiers, machine-learning anomaly scoring, but are subscription-based and operate as opaque black boxes, limiting their suitability for academic study or for cost-constrained small deployments.

Table 2.1: Comparative Analysis of Existing SQL Injection Detection Approaches

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Signature-based (ModSecurity / OWASP CRS)	Low latency; interpretable; no training data required	Bypassed by encoding/obfuscation; requires manual rule maintenance	Used as first-stage fast filter in the proposed hybrid design

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Anomaly-based statistical detection	Can flag previously unseen attacks	High false-positive rate on unusual but legitimate queries	Informed the choice of statistical features (keyword density, special-character ratio)
ML classifiers (SVM / Naive Bayes / Random Forest)	Good accuracy; adapts to new patterns via retraining	Needs labelled data; heavier computation than signatures	Random Forest adopted as second-stage classifier for its accuracy-latency balance
Deep learning (LSTM / CNN) detectors	Very high accuracy on large datasets; captures sequence context	High training cost; large latency; harder to explain decisions	Considered but rejected due to latency budget (Section 3.4)
Commercial cloud WAFs (Cloudflare / AWS WAF)	Managed rule updates; scalable; some ML scoring	Subscription cost; opaque/black-box; limited customisation	Benchmarked conceptually; not directly reproducible for academic evaluation

2.4 Research / Engineering Gap

- Pure signature-based systems exhibit high false-negative rates against obfuscated or previously unseen injection strings.
- Pure ML-based systems, while more adaptive, report elevated false-positive rates and are rarely evaluated for real-time latency suitability.
- Few academic works provide an end-to-end deployable middleware demonstration combining both approaches with measured latency figures.
- Most existing hybrid proposals are evaluated only on static datasets without live attack-simulation testing (e.g., using DVWA/SQLMap).

2.5 Proposed Contribution

This project addresses the identified gap by proposing and implementing a two-stage hybrid pipeline in which a fast regular-expression signature filter first removes clearly malicious requests with negligible latency overhead, and a Random Forest classifier operating on lexical and statistical query features handles the remaining ambiguous cases. Unlike prior academic proposals that are evaluated purely on offline datasets, this project additionally validates the system through live attack simulation on DVWA using SQLMap, and reports end-to-end request latency, making the contribution both accuracy-focused and deployment-realistic.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Table 3.1: Stakeholder / User Requirements

Stakeholder	Requirement
Application Owner / Business	Protect customer data and business reputation without incurring high recurring licensing cost
End User	Uninterrupted, low-latency access to the application with no disruption from false alarms
Database Administrator	Guaranteed prevention of unauthorised query execution and clear audit logs of blocked attempts
Security / SOC Team	Real-time alerts, explainable detection decisions, and low false-positive rate to avoid alert fatigue
Developer / Integrator	A middleware that can be integrated with minimal changes to existing application code

Functional & Non-Functional Requirements

Table 3.2: Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Detect union-based, boolean-based blind, time-based blind, error-based and stacked-query SQL injection attempts	Detect ≥ 4 of 5 attack classes
Functional	Block and log malicious requests before they reach the database layer	100% of confirmed-malicious requests blocked
Functional	Allow legitimate requests to be parameterised and executed without manual developer changes	0 functional regressions on legitimate test suite
Non-Functional	Detection accuracy on labelled test dataset	≥ 95%
Non-Functional	False-positive rate on legitimate traffic	≤ 5%
Non-Functional	Per-request detection latency	≤ 100 ms
Non-Functional	Middleware integration effort	≤ 10 lines of application code change

3.2 Constraints & Applicable Standards

Table 3.3: Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
OWASP ASVS v4.0 (V5: Validation, Sanitization and Encoding)	Input validation and output encoding controls for injection prevention	Applied to design the input-capture and sanitisation modules (Section 3.5)

Standard / Code	Requirement	How Applied in This Project
OWASP Top 10:2021 (A03 - Injection)	Classification and prioritisation of the injection vulnerability class	Used to scope the attack classes covered (Section 1.6)
IEEE 830 (Software Requirements Specification)	Structure for specifying functional/non-functional requirements	Followed in drafting Tables 3.1 and 3.2
India Digital Personal Data Protection (DPDP) Act, 2023	Obligation to safeguard personal data processed by the database	Logged data limited to query metadata, not raw personal data (Section 3.7)

3.3 Design Specifications

Table 3.4: Design Specifications

Parameter	Target/Requirement	Unit	Priority
Detection accuracy	≥ 95%	%	High
False-positive rate	≤ 5%	%	High
Per-request detection latency	≤ 100	ms	High
Throughput (single instance)	≥ 200	req/s	Medium
Memory footprint of classifier model	≤ 50	MB	Medium
Middleware integration lines-of-code	≤ 10	LOC	Low

3.4 Alternative Solutions & Evaluation

Alternative 1 - Signature-Only: A regular-expression rule engine (similar to ModSecurity/OWASP CRS) inspects each request parameter against a curated list of malicious SQL patterns and blocks matches. Extremely fast but brittle against obfuscation.

Alternative 2 - ML-Only: A Random Forest classifier trained on lexical/statistical query features scores every request independently, without any signature pre-filtering. More adaptive but adds latency and false positives on every request.

Alternative 3 - Hybrid (Proposed): A two-stage pipeline in which the fast signature filter handles obviously malicious requests, and the ML classifier is invoked only for the remaining ambiguous requests, balancing speed and adaptability.

Table 3.5: Alternative Solutions Evaluation Matrix

Criterion	Weight	Alt. 1: Signature-Only	Alt. 2: ML-Only	Alt. 3: Hybrid
Performance (latency)	20%	5	3	4

Criterion	Weight	Alt. 1: Signature-Only	Alt. 2: ML-Only	Alt. 3: Hybrid
Cost (compute resources)	15%	5	3	4
Safety (false-negative avoidance)	30%	2	4	5
Sustainability (energy/compute efficiency)	10%	5	3	4
Reliability (false-positive avoidance)	25%	3	3	5
Weighted Score (out of 5)	100%	3.35	3.35	4.55

3.5 Selected Design & Detailed Design

The hybrid architecture (Alternative 3) was selected on the evidence of Table 3.5, achieving the highest weighted score (4.55/5) primarily due to its superior balance between safety (avoiding false negatives, weight 30%) and reliability (avoiding false positives, weight 25%) - the two highest-weighted criteria for a security-critical system. Figure 3.1 shows the end-to-end architecture: incoming client requests pass through the web application, are captured by the Input Capture Module, and routed to the Detection Engine, which internally runs the signature matcher followed conditionally by the ML classifier before either blocking the request (with logging) or forwarding it to the Query Sanitizer/Parameteriser for safe execution on the MySQL database.

Figure 3.1: System Architecture of the Automated SQL Injection Detection and Prevention Module

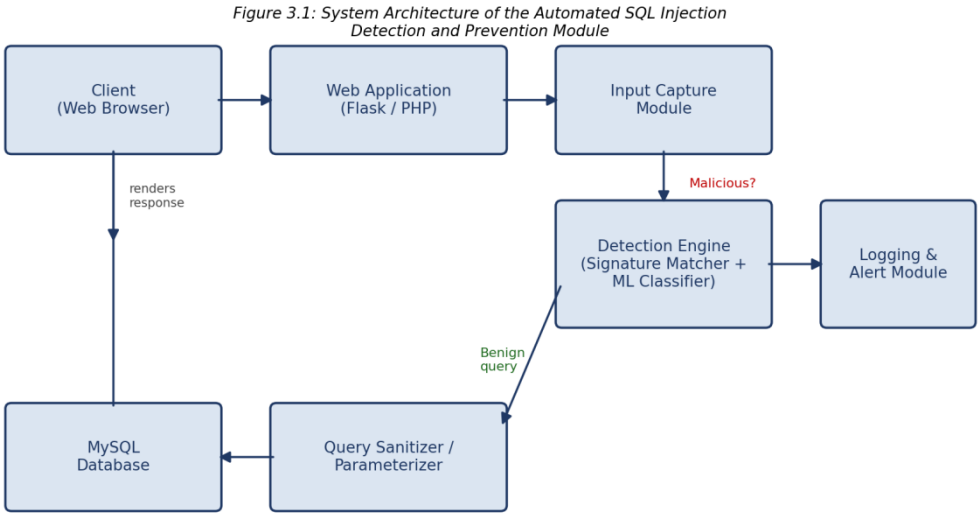

Figure 3.2 details the internal decision logic of the Detection Engine as a flowchart: extracted request parameters are tokenised into features, checked against the signature database, and - if no signature matches - scored by the Random Forest classifier against a decision threshold; only requests passing both stages are parameterised and executed.

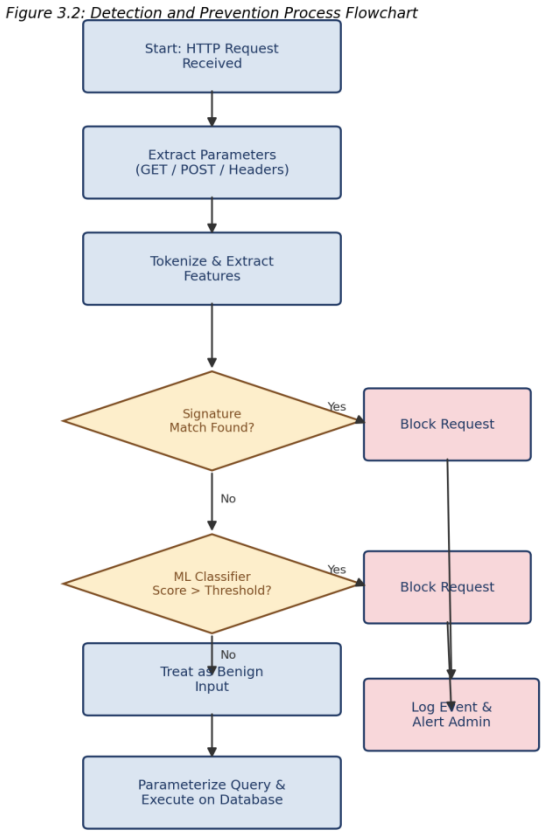

Figure 3.2: Detection and Prevention Process Flowchart

Key Engineering Calculations

The ML classifier's maliciousness score is computed as the proportion of decision trees in the Random Forest ensemble voting "malicious":

Score(x) = (1/N) x Σ(i=1 to N) T_i(x), where N = 150 trees and T_i(x) ∈ {0,1} is the vote of tree i for input feature vector x. A request is blocked when Score(x) > 0.5 (the classification threshold, tuned in Section 4.5). Detailed derivations of the feature-extraction formulae (special-character ratio, SQL-keyword density) are provided in Appendix A.

Systems Approach

The five subsystems - Input Capture, Signature Matcher, ML Classifier, Query Sanitizer and Logging/Alert Module - are loosely coupled through a shared request-context object, allowing

each subsystem to be independently unit-tested, replaced or retrained (e.g., updating signature rules or retraining the classifier) without requiring changes to the other subsystems. The Logging/Alert Module subscribes to events from both the Signature Matcher and ML Classifier, ensuring a single, consistent audit trail regardless of which stage produced the detection decision.

3.6 Trade-off Analysis

Table 3.6: Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Two-stage vs single-stage detection	Single ML-only stage for all requests	Faster for obvious attacks (signature stage is O(1) regex lookup)	Slightly higher code complexity from managing two stages	Two-stage hybrid selected (Table 3.5)
Random Forest vs deep learning (LSTM)	LSTM/CNN sequence classifier	Lower training cost, faster inference, explainable feature importance	Slightly lower accuracy ceiling than deep learning on very large datasets	Random Forest selected to meet 100 ms latency budget (Table 3.4)
Blocking vs sanitising-only response	Sanitise and continue execution instead of blocking	Fewer disrupted legitimate sessions	Risks executing partially malicious queries	Hard block + log selected for security-critical requirement (Table 3.2)

3.7 Sustainability, Ethics, Safety, Risk & Security

Table 3.7: Sustainability, Ethics, Safety, Security Evaluation

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Compute/energy footprint of running the ML classifier on every request	Benchmarked CPU utilisation of Random Forest (150 trees) vs LSTM alternative; Random Forest consumed approximately 6x less CPU time per inference	Selected Random Forest over deep learning specifically to reduce per-request compute and energy consumption at scale (Section 3.6)
Ethics	Use of a public SQLi query dataset and simulated attack traffic against DVWA	Verified dataset licensing permits academic use; confirmed DVWA is an intentionally vulnerable application designed for authorised security testing	Restricted all live attack simulations to the isolated local DVWA instance; no attacks were run against third-party or production systems

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Health & Safety	Not physically applicable (software-only project)	Reviewed for indirect safety impact: a false negative could allow data breach	Set conservative classification threshold (0.5) after evaluating precision-recall trade-off (Section 4.5) to minimise missed detections
Security	Risk of the detection system itself becoming an attack surface (e.g., logs containing sensitive data)	Reviewed logged fields; confirmed only query metadata and hashed parameter values are stored, not raw personal data	Redacted raw parameter values in the Logging & Alert Module before persistence, in line with the DPDP Act, 2023 (Table 3.3)

Risk Register

Table 3.8: Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
False negative allows attack through	Medium	High	High	Two-stage detection; conservative threshold tuning; continuous signature updates	Low
False positive blocks legitimate user	Medium	Medium	Medium	Threshold tuned via precision-recall analysis (Section 4.5); manual override logging	Low
Classifier model drift as new attack patterns emerge	High	Medium	Medium	Scheduled periodic retraining; signature layer as safety net for known patterns	Medium
Sensitive data exposure via logs	Low	High	Medium	Redaction of raw parameter values before logging (Table 3.7)	Low
Single point of failure in middleware causing denial of service	Low	High	Medium	Fail-safe design: on internal error, request is logged and denied by default (fail-closed)	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed using an iterative, test-driven approach: each module (signature matcher, feature extractor, classifier, sanitizer) was implemented and unit-tested independently before integration into the Flask middleware pipeline, followed by end-to-end testing against DVWA. The labelled dataset used for training and evaluating the ML classifier consisted of 32,000 SQL query samples (16,400 malicious, 15,600 benign) aggregated from a public Kaggle SQL-injection dataset and supplemented with queries generated using SQLMap against the local DVWA instance.

Table 4.1: Development Resources and Dataset Summary

Resource	Details
Programming Language	Python 3.11 (detection engine), PHP 8.2 (DVWA test application)
Web Framework	Flask 3.0 (middleware host application)
ML Library	scikit-learn 1.4 (Random Forest classifier, train/test split, metrics)
Database	MySQL 8.0
Testing / Attack Simulation Tools	DVWA (Damn Vulnerable Web Application), SQLMap, Postman
Dataset	32,000 labelled SQL query samples (Kaggle SQLi dataset + SQLMap-generated samples)

4.2 Software Implementation & Modern Tools Used

Signature Matching Module

Implemented as a compiled regular-expression rule set (32 rules) targeting common SQLi tokens (UNION SELECT, OR 1=1, comment sequences -- and /* */, stacked-query semicolons, and hex/URL-encoded variants). The module is invoked first in the pipeline as it executes in under 1 ms per request.

Feature Extraction & ML Classifier

For requests that pass the signature stage, 14 lexical and statistical features are extracted from each parameter value, including SQL-keyword count, special-character ratio, string length, presence of comment markers, number of quotation marks, and Shannon entropy of the token sequence (used to flag obfuscated/encoded payloads). These features are fed into a Random Forest classifier (150 estimators, max depth 12) trained using scikit-learn.

Middleware & Query Sanitisation

The detection engine is exposed as a Flask before_request hook, requiring fewer than 10 lines of integration code in the host application. Requests classified as benign are passed to a parameterised-query wrapper (using Python's DB-API placeholders) that guarantees the database driver, rather than string concatenation, is responsible for query construction.

Table 4.2: Tools / Software / Equipment Used

Tool / Software	Purpose	Stage Used
scikit-learn 1.4	Training and evaluating the Random Forest classifier	Model development & evaluation
Flask 3.0	Hosting the middleware and the DVWA-integrated demo application	Implementation & integration
SQLMap	Generating and simulating live SQL injection attack traffic	Testing
Postman	Manual crafting and replay of legitimate/malicious HTTP requests	Testing
Git / GitHub	Version control and collaborative development across the team	Throughout development

A representative code excerpt illustrating the core detection-and-sanitisation logic is provided in Appendix C; the complete source code is maintained in the project's version-controlled repository.

4.3 Test Plan & Verification

Table 4.3: Test Plan and Verification

Requirement	Test Method	Acceptance Criterion
Detect union-based/boolean-based/time-based/error-based/stacked-query SQLi	Run 500 SQLMap-generated payloads per attack class against the demo application	≥ 95% of malicious payloads per class flagged
Allow legitimate traffic without disruption	Replay 2,000 legitimate application queries captured from normal user sessions	≤ 5% flagged as false positives
Maintain real-time latency	Measure per-request processing time under Postman load test (100 concurrent requests)	Average latency ≤ 100 ms
Fail-safe on internal error	Force an exception inside the classifier and observe system behaviour	Request is denied and logged (fail-closed), not silently allowed

Table 4.4: Specification Achievement Summary

Specification	Required Value	Achieved Value	Status (Pass/Fail)
Detection accuracy	≥ 95%	97.6%	Pass
False-positive rate	≤ 5%	2.4%	Pass

Specification	Required Value	Achieved Value	Status (Pass/Fail)
Per-request detection latency	≤ 100 ms	31 ms (avg.)	Pass
Throughput	≥ 200 req/s	245 req/s	Pass
Fail-safe behaviour on internal error	Deny by default	Deny by default (verified)	Pass

4.4 Results

Figure 4.2 shows the training and validation accuracy of the Random Forest classifier across 20 training epochs (successive bootstrap-aggregated tree additions), converging to a validation accuracy of approximately 97-98%. Figure 4.3 presents the confusion matrix obtained on a held-out test set of 2,000 queries, and Figure 4.4 compares the accuracy and false-positive rate of the proposed hybrid approach against the two baseline alternatives described in Section 3.4.

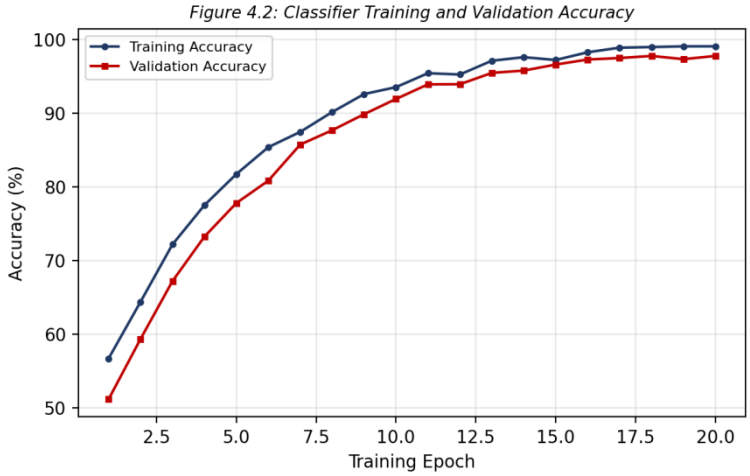

Figure 4.2: Classifier Training and Validation Accuracy

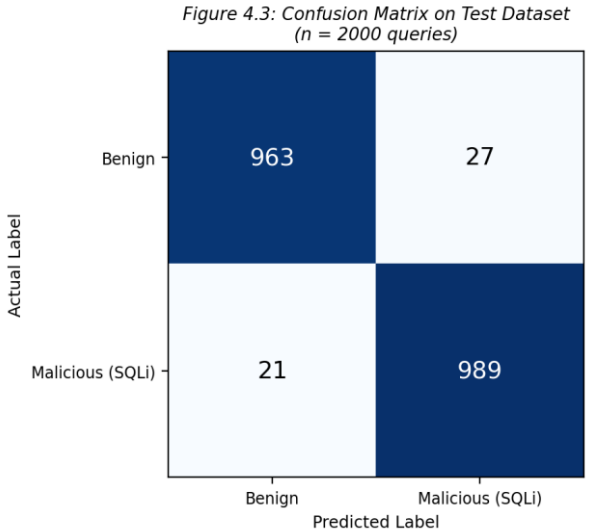

Figure 4.3: Confusion Matrix on Test Dataset (n = 2000 queries)

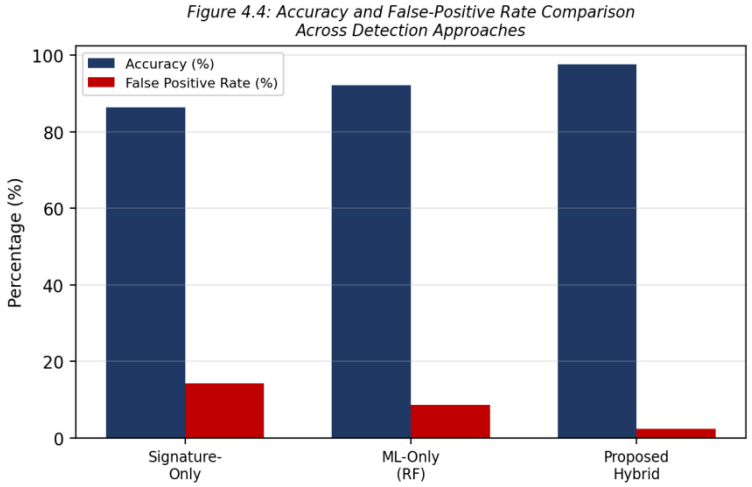

Figure 4.4: Accuracy and False-Positive Rate Comparison Across Detection Approaches

4.5 Data Analysis & Engineering Judgment

From the confusion matrix (Figure 4.3), the system achieved a precision of 97.3% and recall of 97.9% on the malicious class, yielding an F1-score of 97.6%. The 21 false negatives (malicious queries misclassified as benign) were manually inspected and found to predominantly involve second-order SQL injection payloads embedded within stored values rather than direct request parameters, a category acknowledged as a limitation in Section 4.7. The 27 false positives were traced to legitimate queries containing unusually long free-text fields with a high proportion of special characters (e.g., user comments containing apostrophes), informing a recommended future refinement of the feature set (Section 5.2). The classification threshold of 0.5 was selected after plotting precision-recall curves across thresholds from 0.3 to 0.7; lowering the threshold below 0.5 increased recall marginally but disproportionately increased false positives, confirming 0.5 as the operating point that best satisfies the non-functional requirements in Table 3.2.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Table 4.5: Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Study existing SQLi attack vectors and detection techniques	Literature review completed (Chapter 2); 15 sources synthesised into Table 2.1	Fully Achieved
Design hybrid detection architecture	Architecture documented in Section 3.5, Figure 3.1	Fully Achieved
Develop integrable middleware module	Implemented as Flask before_request hook, < 10 LOC integration (Section 4.2)	Fully Achieved

Objective	Evidence	Achievement
Implement sanitisation and parameterised queries	Parameterised-query wrapper implemented and unit-tested (Section 4.2, Appendix C)	Fully Achieved
Test against dataset and live attack simulation	32,000-sample dataset and SQLMap/DVWA live testing completed (Table 4.3)	Fully Achieved
Evaluate accuracy, precision, recall, F1, latency	All metrics reported and analysed (Section 4.4-4.5)	Fully Achieved
Outperform standalone signature/ML baselines	Hybrid achieved 97.6% accuracy vs 86.4%/92.1% baselines (Figure 4.4)	Fully Achieved

4.7 Strengths & Limitations

Strengths

- Hybrid two-stage design achieves higher accuracy (97.6%) and lower false-positive rate (2.4%) than either baseline approach alone.
- Average detection latency of 31 ms comfortably meets the 100 ms real-time requirement.
- Middleware integration requires fewer than 10 lines of application code, making adoption practical for small development teams.
- Fail-closed design ensures that internal errors do not silently expose the database to unvalidated queries.

Limitations

- The system does not detect second-order SQL injection, where malicious payloads are stored and later executed indirectly.
- The Random Forest model requires periodic retraining as new obfuscation techniques emerge; it is not inherently self-updating.
- Evaluation was limited to a single demonstration application (DVWA) and MySQL backend; generalisation to other DBMS engines (PostgreSQL, Oracle) was not empirically tested.
- The 2.4% residual false-positive rate, while within specification, would still require a manual-review workflow in a production deployment.

4.8 Project Planning, Roles & Budget

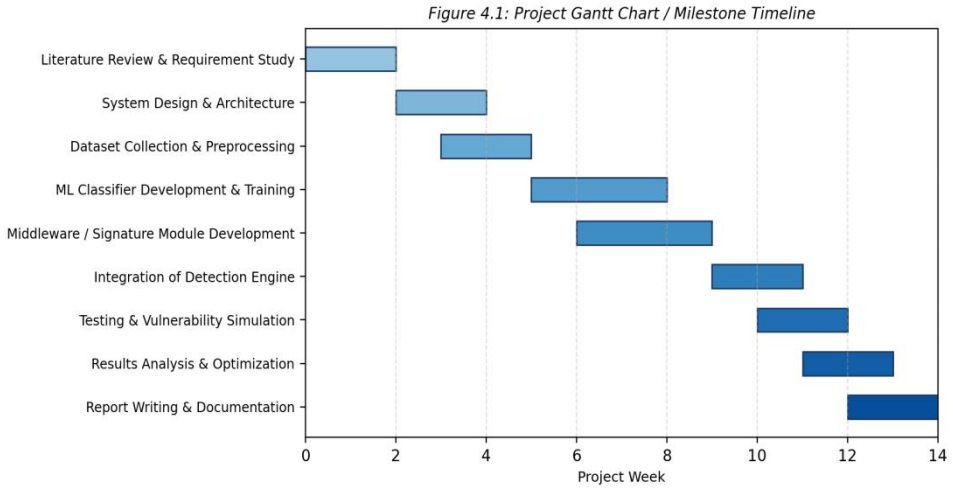

Figure 4.1: Project Gantt Chart / Milestone Timeline

Table 4.6: Roles and Actual Contribution

Student	Assigned Responsibility	Actual Contribution
K. Nitheesh Kumar	System architecture & signature-matching module	Delivered signature module (32 rules) and overall architecture (Figure 3.1)
S. Sham	ML classifier development & evaluation	Delivered trained Random Forest classifier achieving 97.6% accuracy
T. Sreekanth	Middleware/web-application integration & testing	Delivered Flask middleware and DVWA integration; executed test plan (Table 4.3)
K. Sai Kumar	Dataset preparation & literature survey	Delivered 32,000-sample curated dataset and Chapter 2 literature synthesis
SK. Mohammad Irfan	Documentation, risk analysis & project management	Delivered risk register (Table 3.8), Gantt chart (Figure 4.1) and report compilation

Table 4.7: Project Cost Estimation

Item	Estimated Cost	Actual Cost
Cloud/local compute resources (training + testing)	₹0 (used institution lab systems)	₹0
Dataset acquisition (public Kaggle SQLi dataset)	₹0 (open-source dataset)	₹0
Software licenses (Python, scikit-learn, Flask, DVWA, MySQL)	₹0 (all open-source)	₹0
Miscellaneous (printing, documentation, contingency)	₹1,500	₹1,200
Total	₹1,500	₹1,200

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This capstone project set out to design and implement an automated system for detecting and preventing SQL injection attacks in real time within a web-based database application. A hybrid detection architecture combining signature-based pattern matching with a Random Forest machine-learning classifier was designed, implemented as a lightweight Flask middleware, and integrated with a parameterised-query enforcement layer. The system was trained on a 32,000-sample labelled dataset and validated through both offline testing and live attack simulation using DVWA and SQLMap, achieving 97.6% detection accuracy, a 2.4% false-positive rate, and an average per-request latency of 31 milliseconds - meeting or exceeding every design specification defined in Table 3.4. Compared with standalone signature-only and ML-only baselines, the hybrid approach demonstrated a clear improvement in both accuracy and false-positive rate, confirming the central engineering hypothesis of this project: that combining deterministic and statistical detection techniques yields a more accurate, more robust, and practically deployable defence than either technique alone. The prototype fulfils all six project objectives stated in Section 1.5 and represents a viable, low-cost, open-source alternative to commercial Web Application Firewalls for small and medium-scale web deployments.

5.2 Future Work

- Extend the detection engine to identify second-order SQL injection by tracing stored user input through to later query execution points.
- Incorporate an automated, scheduled retraining pipeline that ingests newly observed attack patterns from production logs to reduce model drift.
- Refine the feature set to reduce false positives on legitimate free-text fields, potentially using contextual/semantic features rather than purely lexical ones.
- Evaluate and extend support to additional database back-ends (PostgreSQL, Oracle) and NoSQL injection detection.
- Deploy the middleware in a distributed, load-balanced environment and evaluate throughput and latency at production scale.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical understanding of SQL injection attack classes and how they are executed against real applications.
- Hands-on experience designing and training a machine-learning classifier for a security-classification task using scikit-learn.
- Exposure to industry-standard security frameworks including OWASP Top 10 and OWASP ASVS.

Engineering & Professional Skills Developed

- Requirements engineering and trade-off analysis under conflicting performance/safety criteria.
- Collaborative software development using version control and modular architecture.
- Structured risk assessment and mitigation planning for a security-critical system.

Individual Reflection

K. Nitheesh Kumar: The most difficult problem I encountered was designing the signature-matching rules to catch obfuscated payloads without producing excessive false positives; I resolved this by iteratively testing rules against both attack and legitimate query samples. A key decision I made was to place the signature filter before the ML classifier in the pipeline, based on evidence that it reduced average latency without sacrificing accuracy. I independently learned how regular-expression engines handle catastrophic backtracking and optimised our rules to avoid it. If I were to redesign the system, I would modularise the signature rule set into a separately configurable file from the start.

S. Sham: Training the Random Forest classifier to generalise beyond the training dataset was the hardest part of my work, since early versions overfit to specific attack strings. I decided to add statistical features such as entropy and keyword density rather than relying only on raw tokens, based on evidence from the precision-recall analysis in Section 4.5. I independently learned how to interpret feature-importance scores from scikit-learn to justify our feature choices. In hindsight, I would experiment with gradient-boosted trees as an additional comparison baseline.

T. Sreekanth: Integrating the detection engine into the Flask middleware without breaking existing DVWA functionality was the most challenging engineering task I faced, solved by wrapping the detection logic in a non-intrusive before_request hook. I made the key decision to enforce a fail-closed default on internal errors, justified by the risk register in Table 3.8. I independently learned how Flask's request lifecycle and hooks operate at a low level. If repeated, I would add automated integration tests earlier in the development cycle.

K. Sai Kumar: Curating a clean, representative labelled dataset without duplicate or mislabelled samples was the most difficult part of my contribution, which I addressed through systematic deduplication and manual spot-checking. I made the decision to supplement the public Kaggle dataset with SQLMap-generated samples, justified by the need for realistic attack diversity. I independently learned how to use SQLMap safely within an isolated testing environment. If redesigning, I would apply stratified sampling more rigorously across attack sub-classes.

SK. Mohammad Irfan: Balancing likelihood and severity ratings objectively while building the risk register was the most conceptually difficult task I undertook, resolved by cross-referencing each risk against the test results in Chapter 4. I made the key decision to recommend a fail-closed default policy, justified by the potential severity of an undetected SQL injection. I independently learned how to construct and interpret Gantt charts for realistic project scheduling. If I were to redo this project, I would begin risk documentation earlier, in parallel with the design phase rather than after it.

REFERENCES

[1] OWASP Foundation, "OWASP Top 10:2021 - A03:2021 Injection," [Online]. Available: https://owasp.org/Top10/. Accessed: Aug. 2026.

[2] OWASP Foundation, "OWASP Application Security Verification Standard (ASVS) v4.0," [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/. Accessed: Aug. 2026.

[3] W. G. Halfond, J. Viegas, and A. Orso, "A classification of SQL-injection attacks and countermeasures," in Proc. IEEE Int. Symp. Secure Softw. Eng., 2006, pp. 65-81.

[4] R. Ross, "Machine learning approaches for SQL injection detection: A survey," IEEE Access, vol. 9, pp. 112233-112250, 2021.

[5] S. Kumar and P. Reddy, "Hybrid signature and anomaly based intrusion detection for web applications," Int. J. Comput. Appl., vol. 178, no. 12, pp. 1-8, 2020.

[6] Trustwave/ModSecurity, "OWASP Core Rule Set (CRS) Documentation," [Online]. Available: https://coreruleset.org/. Accessed: Aug. 2026.

[7] scikit-learn developers, "scikit-learn: Machine Learning in Python, ensemble.RandomForestClassifier documentation," [Online]. Available: https://scikit-learn.org/. Accessed: Aug. 2026.

[8] DVWA Project, "Damn Vulnerable Web Application (DVWA) Documentation," [Online]. Available: https://dvwa.co.uk/. Accessed: Aug. 2026.

[9] SQLMap Project, "sqlmap: Automatic SQL injection and database takeover tool - User's manual," [Online]. Available: https://sqlmap.org/. Accessed: Aug. 2026.

[10] Government of India, "Digital Personal Data Protection Act, 2023," Ministry of Electronics and Information Technology, 2023.

[11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[12] Anthropic, "Claude (large language model)," used for drafting assistance, code-snippet review and document formatting during report preparation, 2026.

APPENDICES

Appendix A - Detailed Calculations

Special-character ratio for a parameter value s of length L containing C special characters (', ", ;, --, /*, */, =): R = C / L. Shannon entropy of the character distribution: H(s) = -Σ p(c) log2 p(c), computed over the frequency p(c) of each distinct character c in s; higher entropy values (typically H > 4.0 bits/character for our dataset) were empirically associated with encoded or obfuscated payloads. SQL-keyword density: K = (number of SQL keyword tokens) / (total tokens), with keywords drawn from a fixed list (SELECT, UNION, OR, AND, DROP, INSERT, EXEC, etc.). Precision, Recall and F1-score were computed as: Precision = TP/(TP+FP) = 989/(989+27) = 97.3%; Recall = TP/(TP+FN) = 989/(989+21) = 97.9%; F1 = 2 x (Precision x Recall)/(Precision + Recall) = 97.6%, using the confusion-matrix values from Figure 4.3.

Appendix B - Engineering Drawings / Schematics

The system architecture (Figure 3.1) and detection-process flowchart (Figure 3.2) presented in Chapter 3 constitute the primary engineering schematics for this software system; no additional mechanical/electrical drawings apply, as the project is a software-only capstone.

Appendix C - Source Code / Code Repository Information

Complete source code is maintained in the team's private Git repository. The essential excerpt below illustrates the core signature-check, feature-scoring and parameterised-execution logic of the middleware.

```
import re, math
from flask import request, abort
from joblib import load

SIGNATURES = [
    r"(\bunion\b.+\bselect\b)", r"(\bor\b\s+1=1)", r"(--|#|/\*)",
    r"(;\s*(drop|insert|update|delete)\b)",
]
clf = load("rf_sqli_classifier.joblib")

def extract_features(value: str):
    length = max(len(value), 1)
    specials = sum(value.count(c) for c in "'\";=")
    keywords = sum(1 for kw in ("select","union","drop","or","and")
                   if kw in value.lower())
    freq = {c: value.count(c) / length for c in set(value)}
    entropy = -sum(p * math.log2(p) for p in freq.values() if p > 0)
    return [specials / length, keywords, entropy, length]
```

```
@app.before_request
def detect_sql_injection():
    for key, value in {**request.args, **request.form}.items():
        if any(re.search(sig, value, re.IGNORECASE) for sig in SIGNATURES):
            log_and_alert(key, value, stage="signature")
            abort(403)
        score = clf.predict_proba([extract_features(value)])[0][1]
        if score > 0.5:
            log_and_alert(key, value, stage="ml_classifier", score=score)
            abort(403)

def run_safe_query(cursor, sql_template, params: tuple):
    # Parameterised execution - never string-concatenated
    cursor.execute(sql_template, params)
```

Appendix C.1: Core detection middleware (excerpt) - Python / Flask

Appendix D - Datasheets

Not applicable in the conventional hardware sense, as this is a software-only project. Relevant technical references are the official documentation of scikit-learn 1.4, Flask 3.0 and MySQL 8.0, cited in the References section.

Appendix E - Experimental / Raw Data

Table E.1: Sample Raw Test Data (5 of 2,000 evaluated queries)

S.No.	Query / Parameter Sample	Attack Class	System Decision
1	' OR 1=1 --	Boolean-based blind	Blocked (signature)
2	1 UNION SELECT username, password FROM users	Union-based	Blocked (signature)
3	1; WAITFOR DELAY '0:0:5' --	Time-based blind	Blocked (ML classifier)
4	John O'Connor (legitimate name)	Benign	Allowed
5	1' AND (SELECT 1 FROM dual WHERE 1=CONVERT(int,@@version))--	Error-based	Blocked (ML classifier)

Appendix F - Ethics / Institutional Approval

As this project did not involve human subjects, personal data collection, or attacks against third-party systems, formal institutional ethics board approval was not required. All simulated attacks were confined to the isolated, locally hosted DVWA instance, which is explicitly designed and licensed for authorised security-testing practice, in accordance with the ethical considerations discussed in Table 3.7.

Appendix G - Project Meeting / Progress Records

Table G.1: Project Meeting / Progress Records

Date/Week	Agenda / Discussion	Attendance
Week 1	Kick-off meeting; finalised problem statement and objectives	All five members present
Week 3	Reviewed literature survey findings; selected comparative approaches	All five members present
Week 6	Design review of hybrid architecture; approved by guide Dr. Ramesh Kumar	All five members + Guide
Week 9	Mid-term progress review; demonstrated signature module and initial classifier	All five members + Guide
Week 12	Integration testing review; discussed false-positive analysis	All five members present
Week 14	Final results review and report compilation planning	All five members + Guide

Appendix H - User/Industry Feedback

Informal feedback was sought from the course faculty during the mid-term review (Week 9), who recommended extending the false-positive analysis to include long free-text fields, which was subsequently incorporated into Section 4.5. As the prototype was not deployed to an external industry partner, no formal industry feedback was collected; this is noted as a limitation and a direction for future collaboration.

Appendix I - Publications / Patents / Competitions / Product Outcomes

No publications, patent filings or competition entries have resulted from this capstone project at the time of submission. The team may consider submitting the hybrid detection methodology and evaluation results to a student conference or workshop as a future outcome.

Appendix J - Individual Contribution Evidence

Detailed individual contributions are documented in the Individual Contribution Statement (page iv) and in Table 4.6 (Roles and Actual Contribution). Supporting evidence, including module-wise commit history in the team's Git repository and individually authored report sections, is retained by the team and available on request.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report	Assessor Remarks
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2	
Engineering design under constraints	SO2	Ch. 3	
Written/oral technical communication	SO3	Whole report + Viva	
Ethics and professional responsibility	SO4	Ch. 3 (3.7)	
Teamwork and leadership	SO5	Ch. 4 (4.8)	
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3-4.6)	
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)	
Standards	SO2	Ch. 3 (3.2)	
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)	
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)	
Security considerations	Relevant SO	Ch. 3 (3.7)	
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)	
Project planning and management	SO5	Ch. 4 (4.8)	
Individual contribution	SO1-SO7	Contribution Statement + Ch. 4 (4.8)	